# Comparative Analysis of Human, LLM-Based, and Reinforcement Learning Agents for Autonomous Driving in a Simulated Environment

## Executive Overview and Research Delimitations

The pursuit of fully autonomous driving systems has historically been dominated by modular, rule-based pipelines and, more recently, by end-to-end deep learning architectures. However, the rapid emergence of multimodal Large Language Models (LLMs) and advanced Reinforcement Learning (RL) techniques has fractured the developmental landscape into distinct paradigms of artificial intelligence. The current research ecosystem presents an unprecedented opportunity to conduct a rigorous, controlled comparative evaluation of three fundamentally different decision-making intelligences—biological intelligence (the human operator), language-model-based decision intelligence (the LLM agent), and experience-based learned control (the RL agent)—operating within an identical simulated laboratory.

The core premise of this research asks a fundamentally simple yet structurally complex question: How do a human, an LLM-based agent, and an RL agent perform when given the exact same driving task within the exact same simulated environment? To answer this, the simulator is treated strictly as a neutral test laboratory. The environmental conditions, the physics engine, the vehicular dynamics, and the specific navigational tasks remain absolutely constant. The only variable introduced into the system is the driver—the decision-making engine interfacing with the vehicle's controls.

To maintain academic rigor and prevent uncontrolled scope expansion, it is critical to explicitly define the project's delimitations. This research effort is entirely focused on the implementation, training, and comparative evaluation of driving policies. Consequently, the development of underlying simulation infrastructure is strictly out of bounds. The project will not involve building a new game engine, a physics engine, a road simulator, a 3D world, a vehicular dynamics model, a new reinforcement learning framework, a novel foundational LLM, or a bespoke autonomous driving simulator. Instead, the focus is exclusively directed toward leveraging an existing driving simulator, integrating existing environment APIs, conducting RL training, defining complex multi-objective reward functions, engineering an LLM decision layer, facilitating human baseline experiments, and executing comprehensive statistical analyses. This constrained focus ensures that the research contribution lies in the controlled comparative evaluation rather than in redundant software engineering.

The ultimate research contribution is defined as a controlled comparative evaluation of human, LLM-based, and reinforcement-learning driving agents performing predefined driving tasks within a common simulated environment. If distilled to a single descriptive statement, the project leverages an existing driving simulator to train a reinforcement-learning agent for

predefined driving tasks, subsequently comparing its performance against an LLM-based driving agent and a human driver under identical scenarios. Because the learned policy remains a central, highly engineered component of the architecture, the study firmly satisfies the requirements of advanced reinforcement learning research while simultaneously pioneering intersections with cognitive language modeling and human-in-the-loop benchmarking.

# Theoretical Underpinnings of the Three-Way Comparison

The comparative architecture is designed to evaluate three distinct approaches to dynamic control, each representing a different philosophy of decision-making and environmental interaction. These three pillars—biological, reasoning-based, and experience-based intelligence—are subjected to the exact same tasks and evaluation criteria to reveal their inherent strengths and structural vulnerabilities.

## Experience-Based Learned Control: The Reinforcement Learning Agent

Reinforcement learning relies on the Markov Decision Process (MDP) framework, where an agent interacts with an environment in discrete time steps, receiving state observations and numerical rewards to optimize a cumulative return function. In the context of continuous vehicular control, the RL agent represents a purely reactive, optimization-driven intelligence. The fundamental control loop requires the agent to ingest a state observation from the simulator, process it through a trained neural network policy, output a continuous action space vector (typically steering, throttle, and braking values), and receive a subsequent state and reward signal to drive the policy update mechanism[1].

For complex autonomous driving tasks, state-of-the-art continuous control algorithms such as Soft Actor-Critic (SAC) and Proximal Policy Optimization (PPO) have demonstrated significant success[1]. Advanced frameworks often support SAC natively, alongside extensions like Randomized Ensembled Double Q-Learning (REDQ), which enhance sample efficiency in high-dimensional continuous action spaces[1]. The RL agent is entirely devoid of common-sense reasoning; it learns the physics of the environment and the requirements of the task exclusively through trial, error, and the gradient descent of the reward function. This makes the design of the reward function the single most critical engineering task in the RL pipeline, as the agent will aggressively exploit any mathematical loopholes to maximize returns—a phenomenon known as reward hacking.

## Language-Model-Based Decision Intelligence: The LLM Agent

The integration of Large Language Models into the autonomous driving stack represents a paradigm shift toward reasoning-based, highly interpretable decision-making. Recent architectural frameworks, such as DiLu, DriveMLM, and the Hierarchical Multi-Agent System for Autonomous Driving (AD-H), conceptualize the LLM not as a low-level steering controller, but as a high-level cognitive planner[3]. These Vision-Language-Action (VLA) models excel at processing contextual information, reasoning about edge cases, and emitting human-readable "Chain-of-Causation" traces that explain the logical foundation of their decisions[6].

However, utilizing an LLM for dynamic control introduces a severe latency bottleneck. Pre-trained auto-regressive models are fundamentally incapable of decoding low-level, high-frequency continuous control signals (e.g., adjusting steering angles every 20 milliseconds)[3]. A direct translation of high-level contextual instructions into low-level control signals using an LLM deviates from the inherent language generation paradigm and often results in catastrophic overfitting[3]. Consequently, an effective LLM agent must utilize a hierarchical abstraction. In this paradigm, the LLM ingests semantic environmental data (e.g., current speed, lane deviation, distance to upcoming turns) and outputs discrete, mid-level behavioral commands (e.g., "Maintain speed and prepare for a left turn"). A deterministic lower-level controller, such as a Model Predictive Control (MPC) algorithm or a Proportional-Integral-Derivative (PID) controller, then translates these semantic commands into the continuous steering, throttle, and brake actuations required by the vehicle[3].

### Biological Intelligence: The Human Baseline

Human drivers represent the baseline for biological sensorimotor control, characterized by unparalleled generalized reasoning, spatial awareness, and intuitive physics understanding, yet inherently limited by physiological reaction times, fatigue, and susceptibility to distraction. To effectively benchmark the algorithmic agents, human performance must be captured under the exact same simulated constraints.

A human operator interfaces with the simulator using analog controls, while the system records the driving trajectory at the exact frequency of the algorithmic agents. This telemetry—comprising spatial coordinates, instantaneous velocity, applied steering angles, throttle pressure, braking force, and collision events—serves as the empirical ground truth for adaptability and control smoothness[9]. Comparing the machine-generated trajectories against the human baseline reveals the exact threshold where algorithmic efficiency surpasses biological limitation, and conversely, where biological intuition outperforms algorithmic rigidity.

## Simulation Infrastructure and Environment Selection

The selection of the simulation environment is the most critical infrastructure decision, dictating the feasibility of the entire experimental pipeline. The environment must provide realistic vehicular physics, comprehensive sensor observations, continuous control APIs, and support for both RL training frameworks and human interaction recording. Four primary candidates are rigorously evaluated to determine the optimal test laboratory.

### Option 1: TrackMania Integrated with TMRL

TrackMania, interfaced through the TMRL open-source library, offers a highly optimized, out-of-the-box pipeline designed specifically for deep reinforcement learning in a continuous control setting. TMRL provides a Gymnasium-compatible environment that extracts analog vehicle control and observations such as speed, LiDAR measurements, and raw camera images[1].

The infrastructure utilizes a client-server architecture enabling distributed training. It requires the base TrackMania 2020 game and the Openplanet scripting plugin[11]. The system notably supports Transport Layer Security (TLS) for secure remote training across computing clusters[1]. Actions are defined continuously via virtual gamepad emulation, allowing inputs for gas, brake,

and steering bounded between $-1.0$ and $+1.0$ [1]. The default LiDAR observation shape is heavily structured, typically representing speed, historical LiDAR arrays, and previous actions, which perfectly suits the state requirements of an RL policy[1].

While highly practical and natively supporting advanced algorithms like SAC and REDQ, the environment is fundamentally designed as an arcade racing simulator. Implementing complex urban constraints, such as adhering to speed limits, stopping at precise checkpoints, or navigating intersections, requires extensive custom track design and complex reward restructuring[14]. Furthermore, relying on proprietary game engines introduces dependencies on external updates that can break the Openplanet API, and training requires dedicated NVIDIA GPUs[12].

## Option 2: NVIDIA AlpaSim and AlpaGym Ecosystem

The NVIDIA autonomous driving ecosystem represents the absolute state-of-the-art in closed-loop AV simulation and reasoning-based driver models. The stack includes AlpaSim, an open-source closed-loop simulation framework, and AlpaGym, a high-throughput reinforcement learning framework built directly on top of AlpaSim for training autonomous vehicle policies[15]. Furthermore, Omniverse NuRec allows for the neural reconstruction of driving environments from real-world data[17].

The ecosystem is designed around the Alpamayo foundation models, which are advanced Vision-Language-Action models ranging from 10 billion to 32 billion parameters[6]. These models are natively capable of generating trajectories alongside Chain-of-Causation reasoning traces, bridging the gap between raw perception and high-level planning[6]. However, the hardware constraints are monumental. The default 10B Alpamayo model requires at least one GPU with 24GB or more of VRAM (e.g., RTX 3090, RTX 4090, or A5000), and larger deployments require multi-GPU setups orchestrated via Slurm scheduling on enterprise platforms like Amazon SageMaker HyperPod[6]. While this perfectly aligns with the theoretical combination of LLMs and RL, the sheer computational overhead and the complexity of deploying a microservice architecture makes it prohibitive for highly agile research projects lacking data-center scale hardware.

## Option 3: CARLA Simulator

CARLA is the traditional academic standard for autonomous driving research. Built on Unreal Engine, it utilizes the OpenDRIVE standard to define road networks and provides highly realistic urban environments, dynamic pedestrian traffic, and complex sensor suites[19]. CARLA provides a robust Python API for actor management, vehicle control through Ackermann settings, and sensor data extraction[21].

Despite its visual fidelity, integrating human baselines in CARLA presents severe architectural challenges. Human data collection is typically handled via manual control scripts utilizing the Pygame library[9]. However, documented limitations reveal that running Pygame in a synchronous simulation loop often leads to blocking threads, dropped frames, and system crashes[22]. When manual control is run concurrently with automated scenario runners, it frequently blocks the main execution loop, preventing the scenario from ticking forward unless

carefully orchestrated across separate parallel threads[24]. This instability severely complicates the process of gathering clean, high-frequency human-vs-agent comparative runs.

## Option 4: MetaDrive Simulator

MetaDrive has emerged as an exceptionally strong candidate for comparative autonomous driving research. Built on the Panda3D game engine and the Bullet physics engine, it is designed specifically to support generalizable reinforcement learning rather than pure perception modeling[25].

MetaDrive uses a procedural generation algorithm to create infinite driving scenarios by concatenating block sequences representing straights, curves, intersections, and tollgates[28]. It is highly computationally optimized, capable of running at up to 1,500 frames per second on a standard PC by bypassing the heavy rendering bottlenecks associated with Unreal Engine[27]. The environment natively conforms to the Gymnasium interface, providing comprehensive sensor abstractions including sparse LiDAR arrays, RGB cameras, top-down semantic maps, and direct access to internal state variables such as lane deviation, heading, and speed[27]. Because MetaDrive provides a perfect balance between realistic vehicle dynamics, strict traffic rule enforcement, and ultra-fast RL training throughput, it serves as an optimal test laboratory. Furthermore, it seamlessly supports human keyboard and steering wheel control via built-in, non-blocking manual control scripts[27].

## Environment Evaluation Matrix

To systematically determine the most appropriate infrastructure, the capabilities of the four candidates are evaluated against the specific operational requirements of the research design.

| Evaluation Criteria | TMRL (TrackMania) | NVIDIA (AlpaSim/Alpa Gym) | CARLA Simulator | MetaDrive |
|---|---|---|---|---|
| **Physics and Rendering Engine** | Nadeo Engine (Proprietary) | Omniverse / Custom Microservices | Unreal Engine 4/5 | Panda3D / Bullet Physics |
| **RL Pipeline Compatibility** | Excellent (Native SAC/REDQ) | Excellent (Native AlpaGym) | Moderate (High resource overhead) | Excellent (Native Gymnasium) |
| **LLM Integration Readiness** | Poor (Racing focus lacks semantics) | Excellent (Native Alpamayo VLA) | Moderate (Requires custom API wrapper) | Good (Accessible semantic state variables) |
| **Hardware Infrastructure** | Moderate (Standard | Extreme (24GB+ VRAM | High (8GB+ VRAM | Low (CPU/Basic |

| | | | | |
|---|---|---|---|---|
| **Requirements** | Gaming GPU) | minimum) | minimum) | GPU sufficient) |
| **Human Interface Reliability** | Excellent (Native gameplay) | Poor (Not designed for manual input) | Poor (Pygame synchronous blocking) | Good (Native manual control scripts) |
| **Driving Task Suitability** | High-speed racing and time-trials | Complex urban reasoning and traffic | Urban perception and multi-agent interaction | Behavioral compliance and route following |

Based on the synthesis of computational efficiency, Gymnasium compliance, hardware accessibility, and the ability to dictate precise behavioral tasks, the MetaDrive simulator is identified as the optimal infrastructure for the laboratory environment, with TMRL serving as a viable secondary alternative should continuous-control racing dynamics become a priority. The NVIDIA ecosystem, while theoretically superior, is discarded due to prohibitive infrastructural demands.

# Architectural Abstraction for Fair Evaluation

The most significant technical risk in this comparative analysis is the latency and frequency asymmetry between the Reinforcement Learning agent and the Large Language Model agent. An RL policy, consisting of a lightweight multi-layer perceptron, can easily process observations and emit steering values at high frequencies (e.g., every 20 milliseconds or 50 Hz). Conversely, an LLM, particularly one processing complex visual and semantic data, is substantially slower, often requiring several seconds to autoregressively decode a decision[3]. If this asymmetry is unmitigated, a direct comparison would erroneously declare the RL agent superior simply because the simulation state changes faster than the LLM's decision loop can react.

To ensure a methodologically defensible evaluation, the architecture must implement a fair abstraction layer. Instead of forcing the LLM to output raw steering values continuously, all three agents are evaluated through a standardized Decision API layer.

## The Hierarchical Decision Pipeline

The environment state is processed through a state abstraction module that serializes the physical simulation variables into structured formats. This abstracted state is then fed to the Decision API, which standardizes the input/output expectations for all agents.

For the human agent, analog inputs from steering wheels or gamepads are sampled at the environmental frequency, directly mapping to the vehicle's actuators. For the RL agent, the policy directly receives the continuous numeric vector of the state and outputs the corresponding actuator values.

For the LLM agent, a hierarchical control framework is implemented. Operating at a lower frequency (e.g., 1 Hz), the High-Level LLM Planner receives the semantic state abstraction. It

utilizes its generalized reasoning to output a discrete, mid-level decision. This decision is instantly captured by a deterministic Low-Level Controller (such as a PID loop), which operates at the high environmental frequency (50 Hz). The lower-level controller translates the LLM's high-level intent into the continuous steering, throttle, and brake actuations required by the simulator[8]. This architectural separation ensures that the LLM is evaluated purely on its decision intelligence and reasoning efficacy, rather than being unfairly penalized for inference latency, making the comparison technically precise and academically rigorous.

# Task Definition and Reward Function Mathematics

To effectively benchmark the agents, the research eschews standard racing objectives ("drive as fast as possible") in favor of controlled, predefined behavioral policies. The simulator functions as a rigid testing ground where all agents are tasked with executing a precise navigational sequence.

## The Predefined Navigational Route

The standardized driving scenario dictates that the vehicle is spawned at a starting coordinate and must execute the following sequence of operations:

1. **Acceleration Phase:** Accelerate from a standstill.
2. **Cruising Phase:** Maintain a target velocity of approximately 40 km/h along a straight trajectory while maintaining lane centering.
3. **Maneuvering Phase 1:** Execute a designated left turn at an upcoming intersection.
4. **Deceleration Phase:** Identify a designated deceleration zone and slow the vehicle to 20 km/h.
5. **Precision Stop:** Bring the vehicle to a complete halt exactly at a specified spatial checkpoint.
6. **Maneuvering Phase 2:** Execute a designated right turn.
7. **Final Acceleration:** Accelerate toward the finish line while avoiding any static or dynamic obstacles.

## Engineering the RL Agent's Observation Space

To navigate this sequence, the RL agent relies on a highly structured observation space. The environment will pass specific telemetry to the policy network, including vehicle kinematics (current speed, physical position, heading angle, lateral lane deviation, and distance to the next waypoint). It will also pass environmental awareness parameters, such as distances to road boundaries, classification of upcoming turns, and localized obstacle detection. A central research question within this design is evaluating whether the agent performs better utilizing this structured state information versus raw visual observations[1]. The simulator supports both multi-dimensional arrays for numeric telemetry and 1D sparse LiDAR arrays that measure distances to immediate boundaries without the extreme dimensionality of raw image processing[32]. The exact continuous action space controlled by the RL agent is defined as an array of values mapping to steering (bounded between $-1.0$ and $+1.0$), throttle (bounded between $0$ and $1$), and braking (bounded between $0$ and $1$).

### The Multi-Objective Reward Formulation

Because the RL agent learns exclusively through numerical feedback, defining the reward function is the most critical engineering task. A basic progress reward is insufficient, as it incentivizes reckless speed and corner-cutting. Therefore, a dense, multi-objective composite reward function is formulated to enforce behavioral compliance.

The reward at any given timestep $t$, denoted as $R_t$, is constructed as a weighted sum of behavioral incentives and penalties:

$$R_t = \omega_1 R_{progress} + \omega_2 R_{lane} + \omega_3 R_{speed} + \omega_4 R_{turn} + \omega_5 R_{checkpoint} - \omega_6 P_{collision} - \omega_7 P_{direction} - \omega_8 P_{jerk}$$

Where:

- $R_{progress}$ provides a continuous dense reward for forward movement along the predefined route.
- $R_{lane}$ penalizes the square of the lateral distance from the lane centerline, encouraging strict lane adherence.
- $R_{speed}$ penalizes the absolute error between the instantaneous vehicle speed and the phase-specific target speed (e.g., maintaining 40 km/h).
- $R_{turn}$ and $R_{checkpoint}$ are sparse rewards granted for successfully executing a required maneuver or stopping at the exact coordinate.
- $P_{collision}$ is a massive terminal penalty applied if the vehicle leaves the drivable road surface or intersects with an obstacle.
- $P_{direction}$ penalizes negative longitudinal velocity to prevent the agent from driving backward.
- $P_{jerk}$ penalizes high-frequency oscillations in the action space (e.g., rapidly alternating throttle and brake) to encourage control smoothness and mimic human driving comfort.

The numerical values of the weighting coefficients ($\omega_n$) will be determined experimentally, leading directly into one of the core research questions: investigating how reward design variations fundamentally alter the emergent behavior and overall performance of the trained policy.

# Empirical Evaluation Metrics and Expected Distributions

Evaluating the efficacy of biological intelligence, language-model reasoning, and experience-based control necessitates a multi-dimensional metric suite. A binary success/failure classification is insufficient to capture the nuances of driving behavior. The primary metrics are categorized into task success, navigational precision, safety, efficiency, and control quality.

## Primary Performance Metrics

| Evaluation Metric | Description of Measurement | Empirical Indicator |
|---|---|---|
| **Task Completion Rate** | The percentage of experimental trials where the agent successfully reaches the final destination without triggering a terminal failure state. | Task Success |
| **Success Rate** | A broader measure for the RL portion specifically, calculating the percentage of successfully completed episodes during final evaluation. | RL Performance |
| **Cumulative Reward** | The total integrated reward accumulated by the RL agent over the duration of the episode. | RL Performance |
| **Route-Following Accuracy** | A quantitative measure of how closely the agent followed the prescribed sequence of waypoints and maneuvers. | Navigational Precision |
| **Average Lane Deviation** | The Root Mean Square Error (RMSE) of the vehicle's lateral distance from the mathematical center of the designated lane throughout the episode. | Navigational Precision |
| **Speed Error** | The absolute mathematical difference between the | Navigational Precision |

| | | |
|---|---|---|
| | agent's actual instantaneous speed and the prescribed target speed for that specific route segment. | |
| **Collision Count** | The total number of intersections with environmental hitboxes (obstacles, barriers, other vehicles) per episode. | Safety |
| **Completion Time** | The total elapsed duration required to successfully navigate the route from start to finish. | Efficiency |
| **Control Smoothness** | A calculus-based metric measuring the integral of the jerk (derivative of acceleration) and steering rate of change, identifying aggressive or oscillatory inputs. | Control Quality |
| **Response Latency** | The time elapsed between a discrete environmental state change and the agent's initiation of the correct actuating response. | Cognitive Efficacy |

## Illustrative Final Results Distribution

While exact numerical outcomes will be generated through empirical testing, an illustrative distribution of the expected results highlights the structural differences between the agents. The biological human baseline is expected to achieve near-perfect completion rates and zero collisions, albeit with higher speed error due to the inability to perfectly estimate digital velocity without instrumentation. The LLM agent is expected to exhibit exceptional route accuracy due to its semantic understanding of the instructions, but will likely suffer from higher lane deviation and completion times due to the latency of the hierarchical decision pipeline. The RL agent, once fully converged on an optimal policy, is expected to exhibit the lowest lane deviation and

speed error due to its high-frequency reactive control, but may suffer minor safety infractions if it encounters out-of-distribution environmental states.

| Metric | Human (Biological) | LLM (Reasoning) | RL (Learned Control) |
|---|---|---|---|
| **Completion Rate** | 100% | 82% | 94% |
| **Route Accuracy** | 98% | 83% | 96% |
| **Avg. Lane Deviation** | 0.21 m | 0.48 m | 0.27 m |
| **Speed Error** | 4.2 km/h | 8.7 km/h | 3.1 km/h |
| **Collisions** | 0 | 3 | 1 |
| **Avg. Response Time** | (Baseline) | 1.8 s | 0.02 s |
| **Completion Time** | 95 s | 118 s | 101 s |

*(Note: The data presented in the table above is purely illustrative and serves as a placeholder for the empirical data to be gathered during the experimental phase.)*

# Experimental Framework and Empirical Validation

To elevate the project from a standard software demonstration to a rigorous academic study, a sequence of six highly controlled experiments is designed to isolate variables and definitively answer the core research questions.

## Experiment 1: Baseline Establishment

The initial phase involves running a basic, pre-existing algorithmic policy (such as a hardcoded PID controller following exact waypoints) through the chosen simulator. This experiment establishes baseline performance metrics, verifies the integrity of the physics engine, and validates that the telemetry extraction systems are accurately recording lane deviation, speed, and collisions.

## Experiment 2: Reinforcement Learning Policy Training

This experiment focuses on training the RL agent from a state of randomized weights to a converged policy. The primary output is the generation of a learning curve, plotting the episodic cumulative reward against the total number of training episodes. The objective is to empirically demonstrate that the agent actually learns to navigate the environment over time,

transitioning from chaotic, highly penalized exploration to smooth, high-reward exploitation of the state space.

### Experiment 3: Evaluation of Different RL Configurations

Because continuous vehicular control is highly sensitive to algorithmic architecture, this experiment compares the efficacy of different foundational algorithms. Depending on the exact constraints of the chosen environment, the performance of Proximal Policy Optimization (PPO) will be benchmarked against Soft Actor-Critic (SAC). For environments like TrackMania that natively support it, the framework will also test REDQ-SAC functionality to determine if randomized ensembling improves sample efficiency[1]. The experiment strictly avoids forcing discrete algorithms (like traditional DQN) into environments that demand continuous actuator control.

### Experiment 4: Reward Function Ablation

This experiment directly investigates how mathematical incentive structures dictate emergent behavior. The RL agent is trained under two distinct paradigms. "Reward A" utilizes a basic progress reward, granting positive reinforcement simply for moving forward. "Reward B" utilizes the complex, composite reward function defined earlier, incorporating progress, strict lane adherence, explicit speed control matching, and severe collision penalties. The resulting trajectories of both policies are plotted to visualize how the integration of safety and precision penalties alters the physical behavior of the vehicle.

### Experiment 5: The Three-Way Comparative Trials

This is the core empirical evaluation of the project. The human driver, the LLM-based hierarchical agent, and the fully trained RL agent are subjected to the exact same unseen scenario. Multiple trials are conducted for each agent to ensure statistical significance. The resulting trajectory data is subjected to comprehensive Exploratory Data Analysis (EDA), mapping the spatial paths, speed profiles, and control inputs of all three intelligences to facilitate direct statistical comparison across the primary metrics.

### Experiment 6: Zero-Shot Generalization Evaluation

A well-documented vulnerability of reinforcement learning is its tendency to memorize specific training environments rather than learning generalized task representations[3]. To test this, agents trained exclusively on "Track A" are deployed zero-shot onto a novel "Track B" featuring different curve radii, altered segment lengths, and distinct obstacle placements. This experiment asks a critical research question: Does the agent actually learn the generalized rules of driving behavior, or does it simply overfit and memorize a single track? Existing RL frameworks have explored generalization to unseen tracks, providing a methodological precedent for evaluating the robustness of the learned policy against the semantic adaptability of the LLM and the biological intuition of the human[1].

## Project Execution, Task Distribution, and Timeline

The successful implementation of this highly complex, multi-agent comparative architecture requires stringent project management and a clean division of labor. The development pipeline is distributed across five specialized domains, ensuring that all theoretical and engineering requirements are met while adhering to the explicitly mandated requirement that all members

contribute equally.

## Team Responsibilities and Domain Focus

| Team Member | Primary Responsibility and Domain Focus |
|---|---|
| **Kowshik** | **Simulator and Environment Integration:** Responsible for configuring the chosen simulator (e.g., MetaDrive/TMRL), defining the physical parameters of the environment, establishing the Python API hooks, and ensuring stable state observation extraction. |
| **Shanmukavardhan** | **RL Training and Algorithms:** Responsible for the implementation, hyperparameter tuning, and reward function engineering of the continuous control RL policies (SAC/PPO). |
| **Krishna Surya** | **LLM Agent and Experiment Design:** Responsible for architecting the hierarchical decision pipeline, prompting the VLA models, integrating the low-level deterministic controller, and structuring the overarching experimental methodology. |
| **Junaid** | **Human Experiments and Data Collection:** Responsible for configuring the analog human interface, designing the telemetry recording systems, overseeing the human baseline trials, and aggregating the raw trajectory outputs. |
| **Valijan** | **Evaluation, Visualization, and Statistical Analysis:** Responsible for the Exploratory Data Analysis (EDA), metric calculation, trajectory mapping, and the synthesis of the empirical data into comparative visualizations. |

Despite these specialized focal points, the architectural dependencies demand that all members understand the entire end-to-end pipeline to facilitate effective peer evaluation and system integration.

### Procedural Timeline and Milestone Deliverables

The research execution is strictly aligned with a four-stage review schedule, ensuring incremental validation of the architecture and empirical findings.

- **Review 1 (September 18):** Focuses on the foundational theoretical framework. Deliverables include the finalized problem statement, academic motivation, explicit objectives, comprehensive literature review of current end-to-end driving architectures, the final selection of the simulation environment, and the proposed system architecture diagram.
- **Review 2 (October 23):** Focuses on infrastructure validation and initial data generation. Deliverables include a fully functional simulation environment, an operational RL training pipeline demonstrating initial learning curves, a working prototype of the hierarchical LLM pipeline, the finalized experimental setup, initial data collection, and preliminary Exploratory Data Analysis including preprocessing and feature extraction from the recorded trajectories.
- **Review 3 (November 13):** Focuses on empirical validation and comparative synthesis. Deliverables include the completed RL training regimens, finalized human baseline experiments, finalized LLM generalization experiments, comprehensive metric evaluation, detailed results synthesis, academic discussion of the structural differences observed, and propositions for future work.
- **Final Submission (November 16):** The culmination of the research. Deliverables include the fully formatted, exhaustive research report, the entire compiled codebase submitted as a compressed archive, and a comprehensive README documentation explaining the procedures required to independently execute the code and replicate the findings.

## Strategic Conclusions and Implications

The proposed comparative analysis serves as a highly structured investigation into the fundamental nature of algorithmic control within dynamic environments. By strictly isolating the environmental variables and standardizing the simulation infrastructure, the study transforms a traditional software engineering exercise into a rigorous cognitive science and machine learning evaluation.

The Reinforcement Learning agent represents the apex of reactive, optimized continuous control. Assuming meticulous reward function engineering, it is expected to yield the smoothest trajectories, the lowest latency, and the highest adherence to targeted speed profiles. However, its architectural reliance on numerical optimization highlights its vulnerabilities: poor zero-shot generalization to novel topological structures and an extreme sensitivity to the manually constructed incentive parameters, which often result in unintended behavioral exploitation.

Conversely, the Large Language Model agent represents the frontier of semantic, generalized

reasoning. While structurally disadvantaged in micro-second reaction times—necessitating a complex hierarchical abstraction to interface with low-level vehicle kinematics—its capacity to parse situational context, deduce rules from natural language, and generalize to entirely novel scenarios without iterative retraining positions it as a highly resilient high-level cognitive planner. The hierarchical architecture necessary to deploy the LLM effectively mirrors the future trajectory of enterprise autonomous vehicle stacks, bridging the gap between foundational reasoning models and optimal control theory.

Ultimately, the biological human baseline anchors these algorithmic approaches to reality, providing a definitive standard for adaptability, intuitive physics, and error recovery. The multidimensional data generated by this research will not merely declare a binary "winner." Instead, it will definitively map the distinct operational envelopes of each intelligence type. This controlled evaluation provides the empirical justification required for the development of future hybrid architectures—systems that fuse the continuous, low-level actuator mastery of Reinforcement Learning with the high-level, generalized, and highly interpretable reasoning capabilities of Large Language Models.

## Works cited


1. trackmania-rl/tmrl: Reinforcement Learning for real-time applications, https://github.com/trackmania-rl/tmrl
2. tmrl - PyPI, https://pypi.org/project/tmrl/0.1.3/
3. AD-H: AUTONOMOUS DRIVING WITH HIERARCHICAL AGENTS, https://openreview.net/pdf/e15ef4c8e8f4e0d2db875b42314bcc25546c73dc.pdf
4. Multi-Agent Autonomous Driving Systems with Large Language, https://arxiv.org/html/2502.16804v1
5. (PDF) DriveMLM: aligning multi-modal large language models with, https://www.researchgate.net/publication/398000174_DriveMLM_aligning_multi-modal_large_language_models_with_behavioral_planning_states_for_autonomous_driving
6. nvidia/Alpamayo-R1-10B - Hugging Face, https://huggingface.co/nvidia/Alpamayo-R1-10B
7. NVIDIA Alpamayo 2 Super: Robotaxi AI | GPUYard, https://www.gpuyard.com/blogs/nvidia-alpamayo-2-super-robotaxi/
8. LVLM-MPC Collaboration for Autonomous Driving: A Safety-Aware, https://www.alphaxiv.org/abs/2505.04980
9. Scripts catalogue - CARLA Simulator, https://carla.readthedocs.io/en/latest/catalogue_scripts/
10. Carla-Trajectory-Tracking-Project - GitHub, https://github.com/ActuallySam/Carla-Trajectory-Tracking-Project
11. Installation - Openplanet, https://openplanet.dev/docs/tutorials/installation
12. tmrl/readme/Install.md at master · trackmania-rl/tmrl - GitHub, https://github.com/trackmania-rl/tmrl/blob/master/readme/Install.md
13. tmrl/readme/install_linux.md at master · trackmania-rl/tmrl - GitHub, https://github.com/trackmania-rl/tmrl/blob/master/readme/install_linux.md

14. GitHub - Anca-Mt/TrackmaniaRL-AI: AI agents for Trackmania using, https://github.com/Anca-Mt/TrackmaniaRL-AI
15. NVlabs/alpasim - GitHub, https://github.com/NVlabs/alpasim
16. LLM info - NVIDIA, https://www.nvidia.com/en-us/solutions/autonomous-vehicles/alpamayo/llm-info/
17. Building an End-to-End Physical AI Data Pipeline for Autonomous, https://aws.amazon.com/blogs/industries/building-an-end-to-end-physical-ai-data-pipeline-for-autonomous-vehicle-3-0-on-aws-with-nvidia/
18. AlpaGym is a reinforcement-learning framework for end-to ... - GitHub, https://github.com/NVlabs/alpagym
19. Introduction - CARLA Simulator, https://carla.readthedocs.io/en/0.9.9/start_introduction/
20. CARLA simulator - GitHub, https://github.com/carla-simulator/carla
21. Python API reference - CARLA documentation, https://carla.readthedocs.io/en/latest/python_api/
22. pygame in python samples #1439 - carla-simulator/carla - GitHub, https://github.com/carla-simulator/carla/issues/1439
23. Manual_control.py crashing with rpc server issue #7004 - GitHub, https://github.com/carla-simulator/carla/issues/7004
24. How to run manual_control.py into scenario_manager.py · Issue #554, https://github.com/carla-simulator/scenario_runner/issues/554
25. MetaDrive: Composing Diverse Driving Scenarios for Generalizable, https://www.researchgate.net/publication/361980215_MetaDrive_Composing_Diverse_Driving_Scenarios_for_Generalizable_Reinforcement_Learning
26. Simulation — MetaDrive 0.1.1 documentation, https://metadrive-simulator.readthedocs.io/en/latest/system_design.html
27. MetaDrive Documentation — MetaDrive 0.1.1 documentation, https://metadrive-simulator.readthedocs.io/
28. Environments — MetaDrive 0.1.1 documentation, https://metadrive-simulator.readthedocs.io/en/latest/rl_environments.html
29. Point Maze - Gymnasium-Robotics Documentation, https://robotics.farama.org/envs/maze/point_maze/
30. Getting Start with MetaDrive, https://metadrive-simulator.readthedocs.io/en/latest/get_start.html
31. CARLA Autonomous Driving Simulation with RRT-Based Path, https://github.com/taherfattahi/carla-motion-planning-rrt-based
32. The Impact of LiDAR Configuration on Goal-Based Navigation within, https://pmc.ncbi.nlm.nih.gov/articles/PMC10747335/